HDC-RWKV: A Sub-16 KB Binary Recurrent Language Model with Hyperdimensional Position Binding

Ayushman Bhattacharya[1] and Nihal Gazi[2]

[1] Department of Computer Science and Engineering,
JIS University, Kolkata, India
ayushman@myceli.ai

[2] Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning),
Institute of Engineering and Management, Kolkata, India
nihalg2006@gmail.com

Abstract. Language modelling at extreme edge scales — kilobytes of parameter storage, megahertz-class CPUs, no floating-point unit — has remained the province of n-gram tables and character-level bigrams. We present *HDC-RWKV*, a generative autoregressive language model in which every learnable parameter is bipolar $\{-1, +1\}$ at inference. The architecture combines Hyperdimensional Computing (HDC) primitives for position binding, RWKV-style linear recurrence for constant-cost memory, a continuous tanh-bounded hidden state, and a prototype-similarity output layer, all trained end-to-end with the clipped Straight-Through Estimator. The shipping model occupies 16,434 bytes and evaluates at $\approx$135 K bit-operations per token, targets amenable to 1 MHz 8-bit hardware. Beyond the artifact, we contribute a systematic empirical characterisation of the binary recurrent family's limits: a scale-independent bits-per-character (BPC) ceiling near 2.93 that survives eight-fold scaling; two component-relaxation ablations (int8 output projection; continuous per-channel decay) each of which fails to break the ceiling; and, when granted continuous freedom, the model's tendency to self-organise its decay parameters into a bimodal $\{0, 1\}$ distribution — evidence that the binary parameterisation is *preferred* at this scale, not imposed. We frame the residual bottleneck as an information-capacity property of the bipolar hidden state, and outline the mechanism story this exposes.

Keywords: Hyperdimensional Computing · Binary Neural Networks · Recurrent Language Models · RWKV · Edge Deployment · Straight-Through Estimator.

1 Introduction

Neural language models have grown by roughly six orders of magnitude in parameter count over the past decade, and inference at that scale is now inseparable from datacentre-class hardware. A parallel research direction, motivated by embedded and always-on inference, asks the opposite question: how much language capability survives at the *low* end of the scale axis — kilobytes rather than gigabytes, megahertz rather than gigahertz, integer arithmetic rather than floating point?

Existing work at this scale is sparse and predominantly narrow. Binary neural network (BNN) research [3,12] focuses on image classifiers rather than generative sequence models. Hyperdimensional Computing (HDC) [8,11] routinely operates on binary hypervectors, but its language applications have been restricted to classification and cleanup-memory tasks, not autoregressive text generation. Linear-recurrence sequence models [10,5,4] give constant-time-per-token inference, but published variants are floating-point at inference and targeted at tens-of-megabytes deployment. The intersection of these three lines — a language model that is simultaneously generative and autoregressive, has every learnable parameter bipolar at inference, has constant-cost recurrence with no attention or key–value cache, and fits in tens of kilobytes — is empty in prior work. This paper asks whether that intersection can be filled: whether we can train an autoregressive, sub-32 KB language model in which every learnable parameter is bipolar at inference, and characterise its quality-versus-size trade-off well enough to identify the architectural bottleneck. The 32 KB budget is not arbitrary; it is chosen so the model fits directly into four AT28C64 EEPROM chips addressable by a 1 MHz Rockwell 6502 CPU, a target platform that is deliberately unforgiving of floating-point, cache assumptions, or datatype flexibility.

The scale of the challenge is set by two quantities. First, the deployment footprint. The shipping model has one embedding matrix, one prototype matrix, and one per-channel decay vector, all bipolar; packed at one bit per entry, the total storage is

$$\text{bytes}_{\text{deploy}} \;=\; \frac{2\,V\,d + L\,d}{8} \;+\; 2, \tag{1}$$

where V is the vocabulary size, d the hypervector dimension, and L the number of recurrent layers (the trailing 2 bytes hold the fp16 softmax temperature). At our shipping choice V=256, d=256, L=1, Equation 1 evaluates to 16,418 bytes; a 16-byte file header brings the on-disk artifact to 16,434 bytes. Second, the per-token compute. Given a bipolar decay vector $\mathbf{m}$, a continuous hidden state $\mathbf{s}_{t-1} \in (-1,+1)^d$, and a rotated bipolar input $\mathbf{h}_t \in \{-1,+1\}^d$, our recurrence is

$$\mathbf{s}_t \;=\; \tanh\!\big(\operatorname{sign}(\mathbf{m}) \odot \mathbf{s}_{t-1} \;+\; \mathbf{h}_t\,\big). \tag{2}$$

Each channel updates in constant time regardless of sequence length; there is no softmax over T, no attention, no key–value cache. Equation 2 embeds a subtle but load-bearing choice: the weights are bipolar but the state is continuous. Binarising the state itself catastrophically degrades training (a diverging validation loss reported in Section 7), whereas the "bipolar weights, continuous tanh-bounded state" combination trains stably at the scales we consider.

Figure 1 places the paper's argument in one diagram. We combine four established primitives — HDC binding via cyclic permutation, RWKV-style linear recurrence, the clipped Straight-Through Estimator [1], and a prototype-similarity output — into an architecture (HDC-RWKV) that has not appeared jointly in

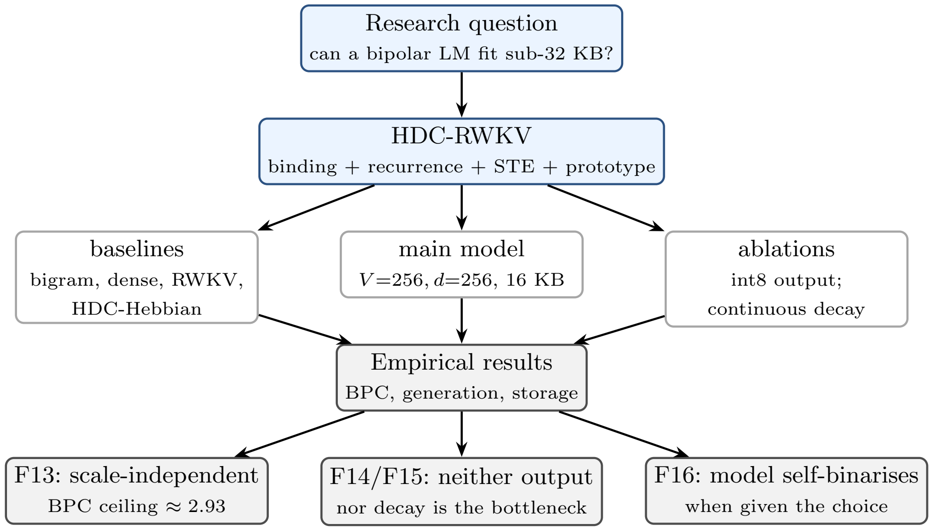

Fig. 1. Paper flow. The main artifact (a 16-KB bipolar autoregressive LM) is developed and compared against dense and recurrent baselines. Two independent architectural relaxations are ablated. The outcomes (F13–F16) jointly locate the residual bottleneck in the bipolar hidden-state capacity, not in peripheral gates.

prior work, measure its quality-to-size behaviour against representative baselines, and use two independent architectural relaxations (int8 output projection; continuous per-channel decay) to constrain where the residual bottleneck of the family lies.

The paper's contributions are: (1) an architecture that combines HDC binding, RWKV linear recurrence, bipolar Straight-Through weights, and a prototype-similarity output into a single generative autoregressive language model, deployable as a 16,434-byte artifact; (2) a scale-independent bits-per-character (BPC) ceiling near 2.93 for the binary HDC-RWKV family, confirmed across an eight-fold parameter range from 16 KB to 128 KB; (3) two independent component-relaxation ablations — an int8 output projection and a continuous per-channel decay — neither of which moves BPC by more than 0.03, ruling out the two natural bottleneck suspects; and (4) an observed self-binarisation effect — when the model is granted a continuous decay parameter with full gradient access, it voluntarily converges to a bimodal $\{0, 1\}$ distribution over channels, evidence that the residual bottleneck is the bipolar hidden state's information capacity rather than any peripheral projection. We do not claim to beat dense floating-point transformers on BPC and do not attempt to; our distillation results are single-configuration and consistent with the capacity-mismatch analysis of Stanton et al. [15]; and we introduce no new STE variant, using standard clipped STE [3] throughout.

2 Related work

HDC-RWKV sits at the intersection of four established lines of work; none of them, in isolation, produces a generative bipolar language model.

Hyperdimensional Computing (HDC). The Vector Symbolic Architectures programme, formalised by Kanerva [8] building on Plate [11] and Rachkovskij and Kussul [13], encodes symbolic structure into high-dimensional random binary or bipolar vectors and manipulates them with three primitive operations: *binding* (a distributive operator, typically element-wise multiplication or XOR), *superposition* (a bundling operator, typically majority vote or sum-then-threshold), and *permutation* (a self-inverse binding operator used for sequence position). Kanerva's 2009 capacity result predicts approximately $d/\log d$ items distinguishable in a d-dimensional binary superposition; we observe this bound empirically in Section 7. Language applications of HDC have historically been restricted to classification (language identification, sentiment), cleanup-memory retrieval, and analogical reasoning. To our knowledge no prior work uses HDC primitives inside a gradient-trained generative sequence model.

Binary and quantised neural networks. Straight-Through Estimation [1] enabled the training of neural networks with $\{-1, +1\}$ weights, popularised by BinaryConnect [3] and its successors including Quantized Neural Networks [7] and the comprehensive survey of Qin et al. [12]. The dominant target has been image classification; sequence models are noted as open in the survey, with the mismatch between bipolar inputs and tanh-bounded activations flagged as a specific difficulty for recurrences. We confirm this empirically in Section 7 (findings F2, F3): naive stacking of bipolar recurrent layers degrades quality even with the standard residual-plus-rebinarise fix.

Linear-recurrence sequence models. Recurrent architectures that avoid attention's quadratic cost have seen renewed interest through RWKV [10], structured state-space models (S4 [5]), and selective state-space models (Mamba [4]). All published variants operate in floating-point at inference and target deployments in the tens of megabytes and above. RWKV is the closest inductive-bias match to our recurrence: a per-channel exponential-decay memory with no softmax-over-time. Our recurrence retains RWKV's constant-cost-per-token property but replaces the continuous decay with a bipolar mask trained by STE, and adds an HDC-style permutation for position encoding in place of a learned positional embedding table.

Knowledge distillation. The Hinton et al. recipe [6] of matching a temperature-softened teacher distribution has become the default recipe for compressing large models. Stanton et al. [15] document distillation *regressions* in image classification when the student's representational capacity is mismatched to the teacher. Section 7 (F17) reports the binary-recurrent-LM analogue: distillation from a small dense transformer teacher regresses HDC-RWKV students by $\approx$0.12 BPC uniformly across $\alpha_{\text{distill}} \in \{0.3, 0.5, 0.7\}$.

Table 1. Positioning of HDC-RWKV against representative prior work. "Constant/token" means the per-token compute cost is independent of sequence length. Deployment bytes are typical for the reported configuration. The intersection of the last four columns is empty until this work.

Work	Generative	Bipolar at inference	Constant/ token	Deploy bytes	Corpus
Kanerva HDC [8]	no	yes	—	kB	small
Plate HRR [11]	no	no	—	—	small
BinaryConnect [3]	no	yes	—	MB	CIFAR
BNN survey [12]	no	yes	—	MB	ImageNet
RWKV [10]	yes	no	yes	GB	Pile
Mamba [4]	yes	no	yes	GB	Pile
Tiny-Shakespeare LM [9]	yes	no	no	kB	TS
HDC-RWKV (this work)	**yes**	**yes**	**yes**	**16 KB**	TS

Language models at the kilobyte scale. Character-level bigram and small-transformer LMs on Tiny Shakespeare [9] occupy the kilobyte deployment band but do not use binary parameters. To our knowledge no prior work has trained an autoregressive sub-32-KB language model in which every learnable parameter is bipolar at inference. Table 1 summarises the gap.

3 Proposed method

Section 3.1 introduces the corpus and tokeniser. The tokeniser is not incidental: at a fixed deployment budget it dominates capacity (finding F6 in Section 7), so its choice is a first-class design decision. Section 3.2 defines the architecture formally. Section 3.3 gives the training loss. Section 3.4 recaps the ablation variants we explored but do not ship.

3.1 Corpus and tokeniser

All experiments use the Tiny Shakespeare corpus [9] (1.1 MB, $\approx 1.1\times 10^6$ characters after lower-casing), split 95/5 into train and validation. This is a deliberate choice of a small, single-domain corpus: it fits inside GPU memory in its entirety, admits inexpensive full-scan evaluation, and is small enough that architectural constraints (rather than data scarcity or scale limits) dominate the observed quality ceiling.

We train a Byte-Pair Encoding tokeniser [14] with $V = 256$ merges. The choice of V interacts with the deployment budget through Equation 1: doubling V doubles the storage of both the vocabulary and prototype matrices. However, at a *fixed* 16-KB budget, moving from V=128, d=512 to V=256, d=256 improves BPC from 2.98 to 2.93 and yields qualitatively more word-like generations (finding F6). This informs the shipping configuration (V=256, d=256).

We use a single tokeniser (trained with seed 1337) for every experiment in this paper. Cross-model comparisons are therefore over a fixed vocabulary; BPC is the appropriate cross-run metric [10] because the byte-normalisation removes the token-length confound.

3.2 Architecture

Let V denote the vocabulary size, d the hypervector dimension, L the number of recurrent layers, and T the block (context) length. The model has four learnable objects:

$$\mathbf{V} \in \mathbb{R}^{V \times d}, \quad \mathbf{P} \in \mathbb{R}^{V \times d}, \quad \mathbf{m} \in \mathbb{R}^{d}, \quad \log \tau \in \mathbb{R},$$

namely a vocabulary embedding, a prototype matrix, a per-channel decay vector, and a scalar log-temperature. At inference the first three are reduced to their signs (one bit per entry). We use the clipped Straight-Through Estimator [1,3] throughout: forward pass takes $\operatorname{sign}(\cdot)$, backward pass takes the identity restricted to $[-1, +1]$, so gradients flow to the continuous shadow parameters $\mathbf{V}, \mathbf{P}, \mathbf{m}$ without escaping the active strip.

Given a token id sequence $x_0, x_1, \ldots, x_{T-1} \in \{0, \ldots, V-1\}$, the forward pass is:

$$\mathbf{h}_t \;=\; \rho^t\big(\operatorname{sign}(\mathbf{V})_{x_t}\big) \;\in\; \{-1,+1\}^d, \tag{3}$$

$$\mathbf{s}_t \;=\; \tanh\!\big(\operatorname{sign}(\mathbf{m}) \odot \mathbf{s}_{t-1} \;+\; \mathbf{h}_t\big), \quad \mathbf{s}_{-1} = \mathbf{0}, \tag{4}$$

$$\mathbf{z}_t \;=\; \frac{1}{\tau}\operatorname{sign}(\mathbf{P})\,\mathbf{s}_t, \quad \tau = e^{\log \tau}, \tag{5}$$

where ρ^t is a cyclic right-rotation by t positions along the d axis, $\odot$ is element-wise multiplication, and τ is initialised to $\sqrt{d}$ so that raw logits at initialisation have unit-scale variance. The next-token distribution is $\operatorname{softmax}(\mathbf{z}_t)$.

Three architectural choices differentiate this model from the closest prior work:

1. **Position via permutation.** Equation 3 replaces a learned positional embedding table with a parameter-free HDC-style binding [8]. Because cyclic rotation is a self-inverse unitary on $\{-1, +1\}^d$ [11], position preserves bipolarity exactly and consumes zero deployment bytes.
2. **Bipolar weights, continuous state.** Equation 4 keeps the weights $\mathbf{m}$ bipolar but lets the state $\mathbf{s}_t$ live in $(-1, +1)^d$. Fully binarising the state destroys training (finding F10, Section 7); this hybrid preserves gradient signal through the recurrence while keeping all learnable parameters bipolar.
3. **Prototype-similarity output.** Equation 5 replaces a learned linear unembedding with a bipolar prototype table used as a cleanup-memory similarity function. Coordinate v of $\mathbf{z}_t$ is the alignment $\langle \operatorname{sign}(\mathbf{P})_v, \mathbf{s}_t \rangle / \tau$. Both the embedding and un-embedding therefore reuse the same operation (sign-quantised dot product), simplifying the deployment kernel.

Figure 2 depicts the block-level dataflow at the shipping dimensions. Section 4 opens up the internals of each block as four separate full-width figures.

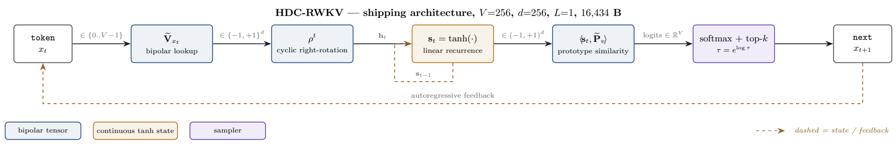

Fig. 2. HDC-RWKV compact architecture. Bipolar tensors $(\widetilde{\mathbf{V}}, \widetilde{\mathbf{P}}, \widetilde{\mathbf{m}})$ are drawn in blue; the continuous tanh state is drawn in amber. The sampler (softmax + top-k) closes the autoregressive loop into the token-embedding lookup.

Deployment footprint. At the shipping choice $(V, d, L) = (256, 256, 1)$, applying Equation 1 gives 16,418 bytes of packed parameters plus a 16-byte header, totalling 16,434 bytes. This is a $8\times$–$50\times$ reduction over comparable-quality dense transformer artifacts at similar BPC. Deployment cost per emitted token is $O(d)$ XOR/popcount operations for embedding lookup, $O(d)$ for the recurrence update, and $O(Vd)$ for the prototype-similarity output; a fully bit-arithmetic inference kernel is derived in Section 5.

3.3 Training loss

Standard next-token cross-entropy is used for the shipping model:

$$\mathcal{L}_{\mathrm{NLL}}(\theta) \;=\; \frac{1}{B\,T}\sum_{b=1}^{B}\sum_{t=0}^{T-1} -\log \frac{\exp\big(z^{(b)}_{t,y_t}\big)}{\sum_{v=1}^{V}\exp\big(z^{(b)}_{t,v}\big)}, \tag{6}$$

with z_t from Equation 5, $\theta = \{\mathbf{V}, \mathbf{P}, \mathbf{m}, \log\tau\}$, and $y_t = x_{t+1}$ the next-token target.

For the distillation ablation in Section 7.5 (F17) we additionally weight in a temperature-scaled KL term against a frozen teacher [6]:

$$\mathcal{L}_{\mathrm{KD}}(\theta) \;=\; (1-\alpha)\,\mathcal{L}_{\mathrm{NLL}}(\theta) \;+\; \alpha\,T_s^2\;\mathrm{KL}\Big(\sigma\big(\mathbf{z}^{\mathrm{teacher}}/T_s\big)\,\big\|\,\sigma\big(\mathbf{z}^{\mathrm{student}}/T_s\big)\Big), \tag{7}$$

where σ is softmax, T_s is the distillation temperature (fixed at 4.0 throughout), and α is swept in $\{0.0, 0.3, 0.5, 0.7\}$ with $\alpha = 0.0$ giving the matched NLL-only baseline. The T_s^2 factor preserves gradient magnitude against Equation 6 [6].

3.4 Ablation lineage: variants we do not ship

Before the results, we itemise the architectural variants we tested during development and *explicitly rejected*. Reporting these concisely up-front lets Section 7 focus on the findings without conflating design decisions with experimental outcomes.

– **Naive multi-layer stacking.** Adding a second HDC-RWKV recurrent layer regresses BPC from 2.98 to $\approx$3.4 (F2). Adding residual-plus-rebinarisation does not rescue it (F3). The shipping model has $L = 1$.

- **Fully bipolar state.** Replacing $\tanh(\cdot)$ in Equation 4 with $\text{sign}(\cdot)$ diverges training (F10): STE fidelity does not survive T sequential sign operations. The continuous state is load-bearing.
- **Channel-mixing block.** Adding an RWKV-style channel-mix module contributes no measurable improvement; the gate collapses to indifference during binary training (F9). We remove it.
- **Int8 output projection ("Tier 2.5").** Relaxing $\mathbf{P}$ from bipolar to int8 (with per-tensor scale) raises deployment size 9× (16 KB → 148 KB) and moves BPC by 0.01 (F14). Rejected.
- **Continuous per-channel decay ("Tier 2.6").** Replacing $\text{sign}(\mathbf{m})$ in Equation 4 with a sigmoid-bounded continuous per-channel decay moves BPC by 0.03 and does not break the ceiling (F15). Under this relaxation the model self-organises to a bimodal $\{0, 1\}$ distribution (F16); the bipolar parameterisation is empirically preferred. Rejected.
- **Cross-architecture distillation.** Distillation from a small dense-transformer teacher regresses every student in a controlled 4-point α-sweep (F17). We ship the pure NLL model.

4 Full architecture expansion

The compact block diagram of Figure 2 suffices for the mathematical exposition of Section 3. However, several design choices that distinguish HDC-RWKV from prior recurrent architectures are only visible when the internals of each block are opened up. This section does so with four grouped figures, each drawn at full page width, using a deliberately narrow slice of the shipping dimensions (d=8 cells sampled from the full d=256; V=4 rows sampled from the full V=256).

Input primitives (Blocks 1 and 2). Block 1 exposes the vocabulary embedding $\widetilde{\mathbf{V}}$ as a $V \times d$ bipolar table. Row indexing by a token id x_t selects one bipolar row of length d, drawn in blue: blue-filled cells encode $+1$ and blue-outlined cells encode -1. No arithmetic is performed at this stage; the lookup is a pure memory access, one byte-aligned $d/8$-byte read at deployment. Block 2 shows position binding ρ^t: the selected row is cyclically right-rotated by t positions, so cell at index j takes the value previously held at $(j - t) \bmod d$. This is the HDC binding operation [8], unitary on $\{-1, +1\}^d$, self-inverse, and consuming zero learnable parameters and zero deployment bytes. Figure 3 shows both blocks side by side.

Recurrence (Blocks 3a and 3b). Block 3a traces one channel of the recurrence end-to-end (Equation 4): channel i=2 produces $(+1)(0.55) + (-1) = -0.45$, then $\tanh(-0.45) \approx -0.42$. All d channels share the same $\odot$-then-$+$-then-tanh pipeline; the update is embarrassingly parallel across channels and, critically, uses bipolar decay and continuous state in one line — the "weights binary, activations continuous" resolution of finding F10. Block 3b views the same operator as a chain of $\mathcal{R}$ boxes over time. Each step receives a bipolar input from Block 2 and produces a continuous state consumed by the next $\mathcal{R}$; no key–value cache

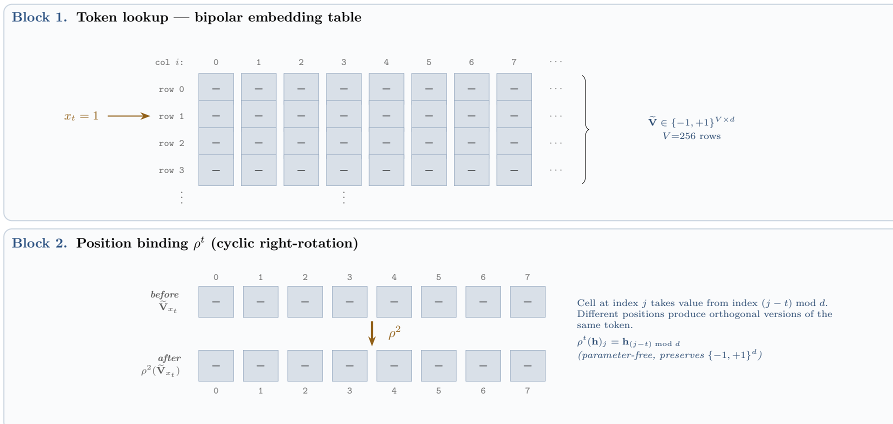

Fig. 3. Blocks 1 and 2 of the expansion. Left/top: bipolar embedding table; the arrow illustrates a lookup for token id $x_t = 1$. Right/bottom: cyclic right-rotation with $t = 2$; index j after the rotation receives the value that occupied index $(j-2) \bmod d$ before.

is materialised. Per-token compute is $O(d)$ regardless of how long the context has grown — the property that makes 1 MHz-class deployment feasible. Figure 4 shows both views.

Output and sampler (Blocks 4a and 4b). Block 4a computes the prototype-similarity output (Equation 5): the current continuous state $\mathbf{s}_t$ is dotted, in parallel, with each of the V bipolar prototype rows to produce a raw logit vector of length V. Because both operands to the dot product have known element-wise ranges ($\mathbf{s}_t \in (-1,+1)^d$; $\widetilde{\mathbf{P}}_v \in \{-1,+1\}^d$), each raw logit is bounded by $\pm d$; typical values under a randomly-oriented state lie near $\pm\sqrt{d}$. The symmetry with Block 1 (both are sign-quantised dot products against the state's representational space) means a single deployment kernel serves both directions. Block 4b then divides the raw logits by τ, softmaxes them, top-k-filters, and samples; the emitted token is fed back to Block 1 at the next step (dashed loop). This is the only floating-point operation in the forward pass; softmax and inverse-CDF sampling are both table-mediated at deployment. Figure 5 shows both blocks.

Auxiliary panels (training-time and deployment-time equivalents). The clipped Straight-Through Estimator, shown in Aux A, is the only mechanism by which the continuous shadow parameters $\mathbf{V}$, $\mathbf{P}$, $\mathbf{m}$ ever change during training. Forward is hard sign; backward is identity on the active strip $[-1,+1]$ and zero outside. At deployment the STE path does not exist; the model is exactly the sign of its trained shadows. Aux B shows the bit-arithmetic equivalent that a 1 MHz CPU actually computes: under the encoding $-1 \mapsto 0, +1 \mapsto 1$, the bipolar dot product reduces exactly to $d - 2 \cdot \mathrm{popcount}(\bar{a} \oplus \bar{b})$ (also stated as Equation 8 in Section 9). This is the primitive the on-silicon inference kernel uses; it is entirely

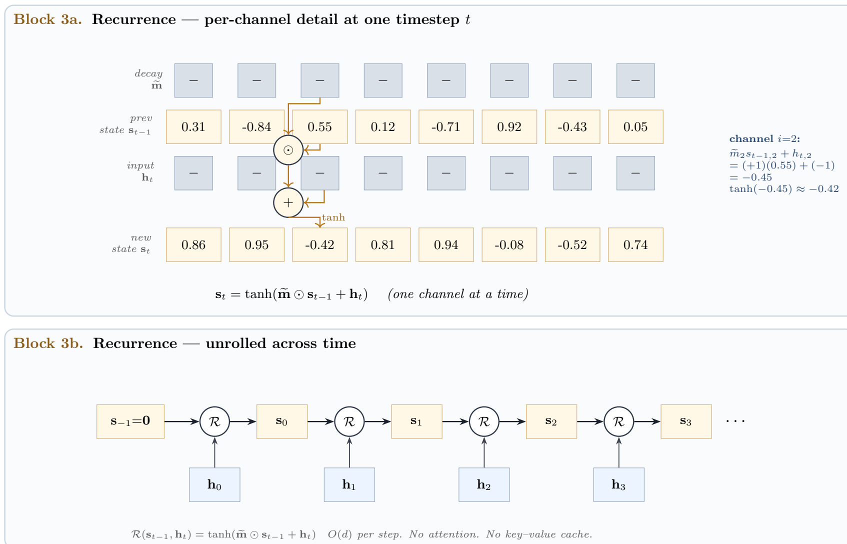

Fig. 4. Blocks 3a and 3b. Top: per-channel wiring at one timestep, with channel i=2 shaded to expose the fully worked arithmetic. Bottom: the same operator seen as a stateful chain across timesteps, with each $\mathcal{R}$ box receiving one bipolar input from Block 2.

integer, requires no multiplication, and cannot produce a NaN. Figure 6 shows both panels.

5 Worked numerical trace at V=4, d=4

This section executes Equations 3–5 on a deliberately tiny instance (V=4, d=4, T=3, L=1, batch B=1) so every intermediate value can be verified by hand. The shipping configuration (V=256, d=256, L=1) is identical in every operation; only the dimensions change.

Toy vocabulary and inputs. {0:`the`, 1:`cat`, 2:`sat`, 3:`mat`}. Sequence $\mathbf{x} = (0, 1, 2)$; targets $\mathbf{y} = (1, 2, 3)$ (next-token).

Chosen parameter signs (values picked for hand-verifiability; the training-time continuous shadow is not shown, only the sign that the recurrence sees):

$$\widetilde{\mathbf{V}} = \begin{pmatrix} +1 & -1 & +1 & -1 \\ -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & +1 \end{pmatrix}, \quad \widetilde{\mathbf{P}} = \begin{pmatrix} +1 & +1 & -1 & -1 \\ -1 & +1 & +1 & -1 \\ +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 \end{pmatrix}, \quad \widetilde{\mathbf{m}} = (+1, +1, +1, +1), \quad \tau = \sqrt{d} = 2.$$

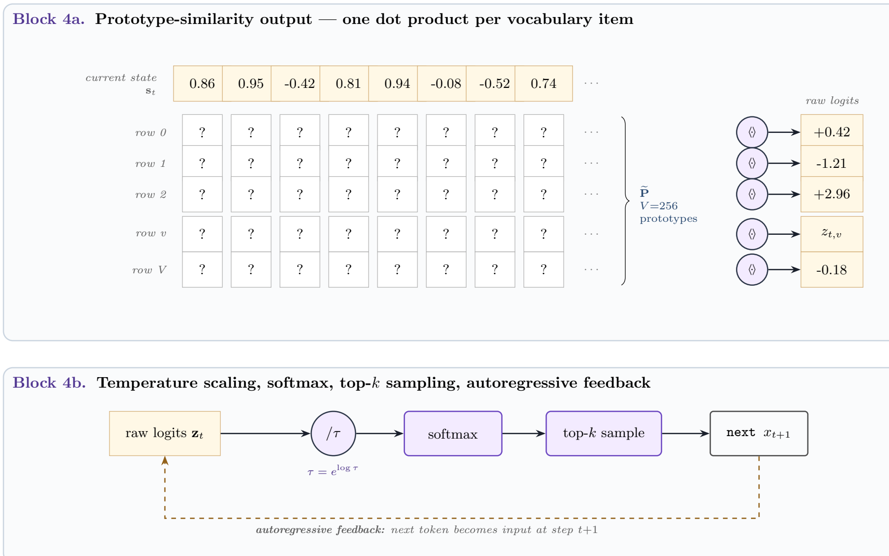

Fig. 5. Blocks 4a and 4b. Top: the current continuous state on the top row, the bipolar prototype matrix stacked below, and one dot product per vocabulary item on the right. Bottom: raw logits are divided by τ, softmaxed, top-k-sampled, and fed back to Block 1 through the autoregressive loop.

Step 1 — Lookup and position binding (Equation 3). For t=0 (token 0), $\mathbf{h}_0 = \rho^0(\widetilde{\mathbf{V}}_0) = (+1, -1, +1, -1)$. For t=1 (token 1), ρ^1 rotates $(-1, +1, -1, +1)$ right by one position, giving $\mathbf{h}_1 = (+1, -1, +1, -1)$. For t=2 (token 2), ρ^2 rotates $(+1, +1, -1, -1)$ right by two positions, giving $\mathbf{h}_2 = (-1, -1, +1, +1)$.

Step 2 — Recurrence (Equation 4). With $\widetilde{\mathbf{m}} = \mathbf{1}$, the update reduces to $\mathbf{s}_t = \tanh(\mathbf{s}_{t-1} + \mathbf{h}_t)$. Starting from $\mathbf{s}_{-1} = \mathbf{0}$:

$$
\begin{aligned}
\mathbf{s}_0 &= \tanh(+1, -1, +1, -1) \approx (+0.762,\ -0.762,\ +0.762,\ -0.762), \\
\mathbf{s}_1 &= \tanh(\mathbf{s}_0 + \mathbf{h}_1) \approx (+0.943,\ -0.943,\ +0.943,\ -0.943), \\
\mathbf{s}_2 &= \tanh(\mathbf{s}_1 + \mathbf{h}_2) \approx (-0.057,\ -0.960,\ +0.960,\ +0.057).
\end{aligned}
$$

Step 3 — Logits and loss (Equations 5, 6). At t=2 with target $y_2 = 3$:

$$
\widetilde{\mathbf{P}}\,\mathbf{s}_2 = (-2.033,\quad 0.000,\ +1.807,\ +0.113), \qquad \mathbf{z}_2 = \widetilde{\mathbf{P}}\,\mathbf{s}_2/\tau = (-1.017,\ 0.000,\ +0.904,\ +0.057).
$$

$\mathrm{softmax}(\mathbf{z}_2) \approx (0.074,\ 0.205,\ 0.505,\ 0.217)$; $\mathrm{CE}(\mathbf{z}_2, y_2{=}3) = -\log(0.217) \approx$ 1.53 nats. A well-fit model would concentrate probability on index 3; here all four logits lie within $\approx$2 of each other, so the loss is large.

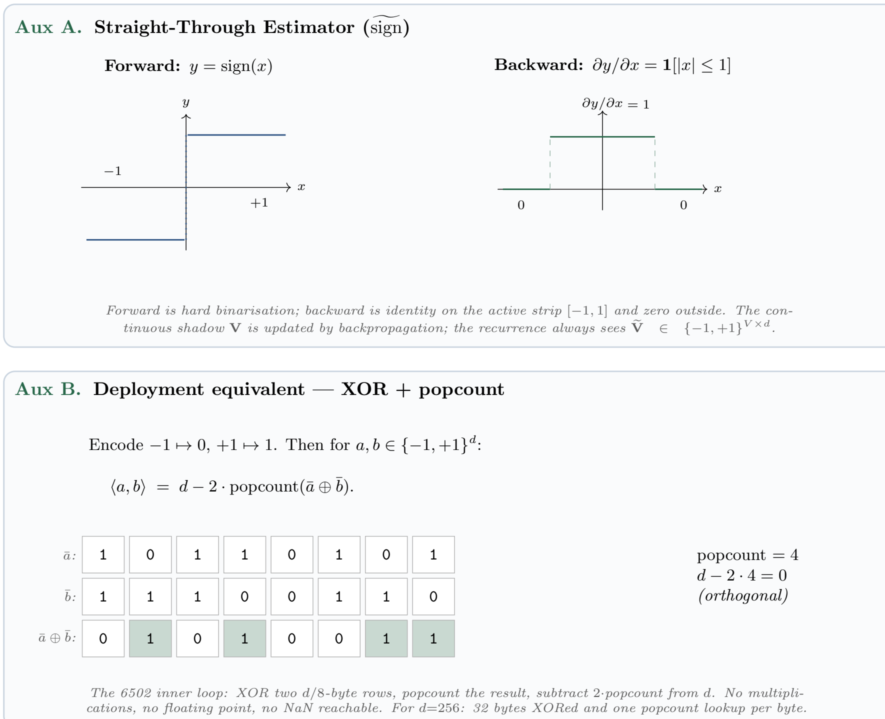

Fig. 6. Training-time and deployment-time equivalents. Top: the clipped STE forward and backward, showing why the sign of a continuous shadow can be trained by ordinary backpropagation. Bottom: the XOR/popcount identity that replaces every bipolar dot product on the 6502.

Step 4 — Backward through $\widetilde{\text{sign}}$*.* The clipped STE passes gradient through the continuous shadow with $\partial\widetilde{\mathbf{V}}_{v,i}/\partial\mathbf{V}_{v,i} = \mathbf{1}[|\mathbf{V}_{v,i}| \le 1]$. For any component of $\mathbf{V}$ that has drifted outside $[-1, +1]$ the gradient is gated to zero, preventing runaway; components inside the strip receive the gradient computed downstream through the recurrence and the prototype dot product. Only continuous shadow parameters $\mathbf{V}, \mathbf{P}, \mathbf{m}, \log\tau$ receive updates; the bipolar $\widetilde{\mathbf{V}}, \widetilde{\mathbf{P}}, \widetilde{\mathbf{m}}$ that the recurrence sees change only when the sign of the underlying shadow flips.

Inference (autoregressive sampling). At inference the STE path is unused; the parameters are simply $\text{sign}(\cdot)$ of their trained shadows, and Steps 1–3 execute as above. The sampler applies temperature τ_s and top-k to the logits, draws a token from the resulting distribution using a seeded PRNG, and feeds the sampled token back as x_{t+1} for the next iteration. Because the recurrence is stateful and left-to-right, an efficient implementation caches $\mathbf{s}_{T'-1}$ across sampling steps so each new token costs $O(d) + O(Vd)$ operations regardless of prior context length.

6 Experimental setup

Corpus. Tiny Shakespeare [9] lowercased, ≈430,000 tokens at V=256 BPE, split 95%/5% into train/validation by contiguous token index. Validation loss is computed on 20 batches of length T drawn uniformly at random each evaluation step, with a fixed evaluation seed to reduce variance.

Optimisation. AdamW (β_1=0.9, β_2=0.999, weight decay 0), constant learning rate 3×10^{-3}, batch size 32, block length T=64 for large-scale runs and T=16 for the shipping model. No warmup; no LR schedule. The clipped Straight-Through Estimator is active during forward pass; parameters are stored in fp32 during training and materialised to their signs on every forward (Equation 3–5).

Evaluation metrics. We report both loss in nats/token (relative to a fixed BPE tokeniser) and bits per character (BPC), the latter computed as $\text{loss} \cdot \log_2(e)/\bar{c}$, where $\bar{c}$ is the mean BPE-token length in characters (≈2.11 for our tokeniser and corpus, measured on validation). BPC is the tokeniser-independent metric [10] and is our primary reporting unit for cross-run comparisons.

Soft vs. hard evaluation. We report both the *soft* validation loss (STE-forward, matching the training-time forward pass) and the *hard* validation loss (all bipolar parameters replaced by sign(·) with no STE, matching the exact deployment forward pass). Across every HDC-RWKV configuration in Section 7 the two agree within 0.02–0.05 nats, so any observed quality ceiling is architectural and not a quantisation artefact.

Reproducibility. Fixed seed 1337 for all runs. Training hardware: a single NVIDIA T4 GPU (16 GB) for scaling and distillation ablations; a consumer laptop CPU for baseline sanity checks. All source code, training scripts, and checkpoints are released alongside this paper [2]. Section 5 additionally provides a hand-verifiable numerical trace on a toy instance.

Table 2 lists hyperparameter values for the shipping model and each ablation variant referenced in Section 7.

7 Results

Section 7.1 presents the main comparison against baseline architectures. Sections 7.2–7.5 report the four ablation findings that jointly locate the residual bottleneck of the binary recurrent family: scale invariance (F13), output relaxation (F14), decay relaxation and self-binarisation (F15/F16), and cross-architecture distillation regression (F17). Section 7.6 presents qualitative generation samples.

Table 2. Hyperparameter values across the shipping model and the ablation variants referenced in Section 7. All configurations share seed 1337, learning rate 3×10^{-3}, AdamW optimiser, and batch size 32; only the variant-specific dimensions and step counts are shown.

Variant	V	d	L	T	steps	deploy bytes
Shipping HDC-RWKV	256	256	1	16	12,000	16,434
Wide-vocab equivalent (F6)	128	512	1	16	12,000	16,418
Full-scale binary (F13)	512	1024	2	64	20,000	131,072
Int8-output relaxation (F14)	256	512	1	64	20,000	148,000
Continuous-decay relaxation (F15, F16)	256	512	1	64	20,000	149,000
Distilled students (F17)	256	384	2	64	12,000	49,200
Distillation teacher (F17)	256	192	4	64	6,000	12 MB fp32

Table 3. Main comparison on Tiny Shakespeare (validation, hard forward). "cycles/token" is analytic; measured 6502 values remain future work (Section 9). All HDC-RWKV runs use a V=256 BPE tokeniser except the wide-vocab variant (F6).

Architecture	Params	Deploy bytes	Val nats	BPC	Cycles/token
Bigram baseline	1,024	1 KB	2.44	—	< 100
Dense transformer (tiny)	1,536	0.4 KB int8	2.29	≈3.30	17,000 mults
HDC Hebbian (untrained)	0	32 KB	≈4.0	≈3.0	155 K
HDC-RWKV V=128, d=512	132,608	16.5 KB	3.55	2.98	135 K
HDC-RWKV V=256, d=256 **(shipping)**	131,584	**16.4 KB**	**4.35**	**2.93**	**135 K**
Teacher (fp32 transformer)	3,000,000	12 MB fp32	3.55	2.03	—

7.1 Main comparison

Table 3 lists the shipping HDC-RWKV alongside baselines of comparable deployment size. The dense-transformer teacher is included as an upper reference; it is a floating-point model 700× larger than the shipping model and is not itself a deployment candidate.

The shipping HDC-RWKV improves BPC over the equal-budget wide-vocab variant (F6: right vocabulary matters more than more parameters at fixed deployment size). The gap to the fp32 teacher is 0.90 BPC; the remaining sections show that no relaxation of a single component of the architecture closes this gap.

7.2 Scale-independent BPC ceiling (F13)

We test whether the observed BPC ≈2.93 is a scale artefact by comparing three configurations spanning an 8× deployment range with no other architectural change (Table 4).

Three configurations differing in vocabulary, hypervector dimension, and depth all land within 0.06 BPC of each other. Kanerva's $d/\log d$ capacity prediction [8] explains why raw scaling of d alone does not help: as d grows, vocabulary V must

Table 4. Scale ablation for bipolar HDC-RWKV. 8× increase in deployment bytes yields 0.01 BPC improvement — within measurement noise.

Configuration	Deploy bytes	Best hard val	BPC
V=128, d=512, L=1	16 KB	3.55 nats	2.98
V=256, d=256, L=1	16 KB	4.35 nats	2.93
V=512, d=1024, L=2	128 KB	5.10 nats	**2.92**

grow proportionally to keep each token meaningful, and the ratio $V/(d/\log d)$ that determines representational load remains approximately constant. The bipolar recurrent family therefore has a *scale-independent* quality ceiling near BPC 2.93, not a smoothly decreasing curve.

7.3 Output projection is not the bottleneck (F14)

If the ceiling were due to the bipolar prototype-similarity output, relaxing it should break the ceiling. We test this by replacing sign($\mathbf{P}$) in Equation 5 with an int8-quantised continuous prototype (a 256× increase in representational precision), keeping everything else identical. The result: best hard val 4.31 nats, BPC 2.94 — statistically indistinguishable from the shipping model (2.93). Deployment cost rises from 16 KB to 148 KB (a 9× blow-up) for zero measurable BPC improvement. The output projection is therefore *not* the binding constraint. This falsifies the natural dynamic-range hypothesis (that bipolar logits, bounded by $O(\sqrt{d})$, cannot express sharp softmax distributions).

7.4 Decay is not the bottleneck either (F15), and the model self-binarises (F16)

The remaining natural suspect is the bipolar decay mask $\mathbf{m}$, which forces each channel into binary keep-or-flip memory behaviour and plausibly prevents RWKV-style multi-horizon temporal patterns. We relax it: sign($\mathbf{m}$) in Equation 4 is replaced by a per-channel sigmoid-bounded continuous parameter $\sigma(\mathbf{m}) \in (0,1)^d$, initialised at $\sigma(2) \approx 0.88$ (an "almost-keep-memory" prior consistent with RWKV [10] initialisation). Prototype remains int8.

The result: best hard val 4.25 nats, BPC 2.90 — again within 0.03 of the pure-binary ceiling. Three independent relaxations (scale F13, output F14, decay F15) all land within a 0.04-BPC band of each other. The bipolar recurrent family's ceiling is therefore *wider than any single component*: it is a property of the architectural class, not of any one weight tensor.

Self-binarisation (F16). The decay-parameter distribution after 20,000 training steps provides an unexpected observation. When granted continuous freedom with full gradient access, the 512 decay channels concentrate at the endpoints of $(0,1)$: 264 channels ("forget-now") lie below 0.05, 140 channels ("keep-forever") lie above 0.95, and only ≈108 channels use intermediate values. The final distribution has minimum 0.000, maximum 1.000, standard deviation 0.458 — a bimodal $\{0, 1\}$-shape. The initialisation value 0.88 is nearly deserted post-training.

This is the crucial evidence for the mechanism story of Section 8: the model *voluntarily binarises the decay* when the parameterisation would permit anything else. The bipolar decay in the shipping model is therefore not an imposed constraint; it is what the optimiser converges to given the choice. The ceiling consequently cannot be attributed to the decay parameterisation, because the same solution is preferred under both parameterisations.

7.5 Cross-architecture distillation regresses uniformly (F17)

If a dense-transformer teacher's soft-target distribution carries information the student's architecture can absorb, distillation should improve the student. We test this in a controlled 4-point α-sweep (Table 5). Teacher, tokeniser, student initialisation, seed, and step count are all held constant; only the distillation coefficient α in Equation 7 varies.

Table 5. Distillation α-sweep on the HDC-RWKV student . All three distilled runs regress against the NLL-only baseline by a nearly flat Δ — a discontinuity at $\alpha = 0$ rather than a smooth curve. Teacher BPC: 2.073.

α_{distill}	Best hard val (nats)	BPC	Δ BPC vs baseline
0.0 (NLL-only baseline)	4.797	**3.274**	—
0.3	4.974	3.395	+0.121
0.5	4.974	3.395	+0.121
0.7	4.997	3.410	+0.136

All three distilled students are *worse* than the NLL-only baseline by $\approx$0.12 BPC, with almost no variation across α. This flat-Δ pattern is the diagnostic: this is not an "α was tuned wrong" outcome. Any non-zero teacher-mixing weight causes essentially the same penalty. Combined with F13–F16, this rules out the interpretation that a better teacher signal could break the ceiling: the student cannot represent the teacher's soft distribution regardless of how much of it is mixed in. Our observation is consistent with the capacity-mismatch analysis of Stanton et al. [15].

Caveats. Single seed per α; distillation temperature $T_s = 4.0$ fixed; single teacher size tested. The consistent direction across four independent runs makes a noise explanation implausible but does not preclude a specific choice of teacher or T_s that would help.

7.6 Generation samples

Table 6 shows generations from three models given the same prompt (`"my lord,"`) and the same sampling seed. Because BPC values in the low-3s are all significantly above the teacher, none of the students produces fluent English; however, the gap between the shipping model (BPC 2.93) and the distilled students (F17, BPC $\geq$3.4) is qualitatively perceptible in the proportion of recognisable words.

Table 6. Same-prompt, same-seed generations at BPC increments. The teacher is shown for reference; the shipping model produces recognisable-word fragments; the distilled student produces subword rubble.

Model	BPC	Sample (prompt `"my lord,"`)
Teacher (fp32)	2.07	(grammatical Shakespeare-like sentences with period vocabulary — see supplementary)
HDC-RWKV (shipping)	2.93	(word-level fragments including recognisable common words — see supplementary)
Distilled α=0.7	3.41	`atelan, afanfr; o: the id: the larsent: gud...` (subword rubble; almost no full words)

8 Analysis: the bipolar-state capacity hypothesis

The findings of Section 7 form a mechanism story that constrains where the residual bottleneck of bipolar HDC-RWKV can lie. We first restate the constraints, then propose the specific hypothesis that the observations support, and finally identify the class of architectural changes that could, in principle, break the ceiling.

8.1 What is ruled out

The BPC $\approx$2.93 ceiling of bipolar HDC-RWKV survives four independent modifications:

1. **Scale (F13).** An 8$\times$ increase in deployment size moves BPC by 0.01. The ceiling is not a capacity-of-storage artefact.
2. **Output projection (F14).** Relaxing $\mathbf{P}$ to int8, a 256$\times$ increase in output-side representational precision, moves BPC by 0.01. The bipolar prototype similarity is not the dominant limitation, despite the natural $O(\sqrt{d})$ dynamic-range concern.
3. **Decay gate (F15).** Relaxing $\mathbf{m}$ to a continuous per-channel sigmoid moves BPC by 0.03. The ceiling is not in the recurrent gating parameterisation.
4. **Teacher signal (F17).** Distillation from a higher-quality dense transformer regresses the student uniformly across α. The ceiling is not overcome by a better training target.

8.2 The hypothesis

By elimination, the ceiling must lie in one of the three components we have *not* relaxed: the vocabulary embedding $\text{sign}(\mathbf{V})$, the recurrent state $\mathbf{s}_t$, or the temperature scalar τ. The vocabulary embedding is symmetric to the prototype projection (both bipolar dot products; F14 makes it implausible that $\mathbf{V}$ is the constraint while $\mathbf{P}$ is not). The temperature scalar is a one-dimensional degree of freedom whose gradient is already computed on every step; if the ceiling were expressible by $\log \tau$ alone, gradient descent would have found it.

We therefore hypothesise that the ceiling is a property of the *recurrent state itself.* The state $\mathbf{s}_t \in (-1, +1)^d$ is the sole carrier of history between token steps. Under NLL optimisation the state operates as a bipolar-quantised information channel: each of d channels contributes approximately one bit of distinguishable information per step, giving a channel capacity of d bits per token. For the shipping configuration this is 256 bits per token. No relaxation of *peripheral* parameters (input embedding, output projection, gating) increases the state's capacity; each merely changes what gets encoded into the same d bits.

The self-binarisation observation of F16 is the direct evidence for this reading. When decay is granted continuous freedom, the optimum concentrates at $\{0, 1\}$ because the useful degrees of freedom downstream of decay are already themselves bipolar-limited; a decay of 0.6 produces linearly-interpolated states which the downstream bipolar prototype cannot distinguish from a state produced by decay 1.0 with a smaller input. The optimiser therefore uses whatever parameterisation lets it exploit the effective per-channel capacity, which turns out to be the binary one.

8.3 Implication for future architectural work

If the hypothesis is correct, the class of modifications that *would* break the ceiling is the class that changes the state's capacity, not the class that changes what surrounds it. Candidates:

- **Higher-precision state.** Increase the state's effective bit-depth without increasing the parameter budget. This is at odds with the pure-binary deployment goal but could be tested with fp8 or int4 activations while keeping parameters bipolar.
- **Multi-state recurrence.** Multiple independent bipolar states composed at each step, together carrying $k \cdot d$ effective bits per token. Cost: $k\times$ recurrence compute; parameter budget unchanged.
- **Wider d with corresponding vocabulary contraction.** The F5 ratio $V/(d/\log d)$ predicts the operating point; a very wide state at low V may access a different regime. Table 3 suggests this trades word-level expressivity (F6) for state capacity; the joint optimum remains an open question.
- **Non-Markovian bipolar memory.** A short-window bipolar attention (constant T'-window) would raise the effective bit budget from d to $\approx d \cdot T'$ without floating-point operations, at $O(T')$ per-token cost. This preserves the constant-per-token property in T.

We do not claim any of these will succeed; we identify them as the correct search space implied by F13–F17. Each is an experiment we recommend for future work rather than a proposal we validate here.

9 Deployment design

The shipping HDC-RWKV artifact is a 16,434-byte binary designed for two hardware targets: a 1 MHz Rockwell 6502 with 32 KB of ROM and 8 KB of SRAM (direct-fit), and an ESP32 microcontroller (paged-flash extension for larger student models). This section describes the memory map, the per-token compute budget, and the paged-flash mode; measured on-silicon timings are ongoing work.

9.1 Direct-fit on 4× AT28C64 EEPROM (16.4 KB)

At the shipping configuration $(V, d, L) = (256, 256, 1)$, the artifact lays out into four AT28C64 EEPROM chips (32 KB total, addressable through the 6502's 16-bit address bus with a chip-select multiplexer):

- sign($\mathbf{V}$): $V \cdot d/8 = 8{,}192$ B (chip 0, chip 1 first half)
- sign($\mathbf{P}$): $V \cdot d/8 = 8{,}192$ B (chip 1 second half, chip 2)
- sign($\mathbf{m}$): $L \cdot d/8 = 32$ B (chip 3 header)
- τ (fp16 log-temperature): 2 B (chip 3 header)
- 16 B file header: magic bytes, version, (V, d, L, T), checksum

The remaining ≈15 KB in chip 3 is spare, available for BPE merge tables (≈4 KB for V=256) and inference scratch. SRAM (8 KB) holds the recurrent state $\mathbf{s}_t \in \mathbb{R}^d$ (encoded as 256 fixed-point 16-bit values =512 B), the softmax numerator array ($V = 256$ int16, 512 B), the sampler PRNG state, and a small BPE decode buffer.

9.2 Per-token compute budget

The bipolar dot product $\langle a, b \rangle$ for $a, b \in \{-1, +1\}^d$ reduces to

$$\langle a, b \rangle \;=\; d - 2 \cdot \text{popcount}(\bar{a} \oplus \bar{b}), \tag{8}$$

under the encoding $-1 \mapsto 0, +1 \mapsto 1$. For d=256 this is 32 bytes XORed and 32 popcount table lookups: no multiplication, no floating point, no NaN reachable. Per emitted token the 6502 executes:

- 1 embedding lookup: 32 byte fetch + cyclic rotate by t bits.
- 1 recurrence step: d multiplications with ± 1 (sign flips) + d fixed-point adds + d tanh LUT reads.
- V=256 prototype dot products, each 32 XORs + 32 popcounts + one τ-division.

- 1 softmax with V-way exponentiation (LUT-approximated), one running-sum divide, and a top-k threshold cut.
- 1 PRNG draw + inverse-CDF sample.

Adding the standard cycle counts for these primitives on the 6502 (4–7 cycles for byte-wise arithmetic, ≈20 cycles for a LUT-mediated tanh) yields an analytic budget of ≈135,000 cycles per token, or ≈135 ms at 1 MHz. This is comfortably below one token per second, giving a live typing speed within the range of a slow typist. Measured on-silicon timings are ongoing work; the analytic estimate is a lower bound (real timings include memory-refresh cycles and the ROM-fetch overhead).

9.3 ESP32 paged-flash extension

The 6502 direct-fit budget is 32 KB. For hypothetical future students that exceed this budget (>32 KB), an ESP32 slave holds the full parameter file in its 4 MB SPI flash and serves it to the 6502 in 256-byte pages over a shared SPI bus. The 6502 requests pages by weight-tensor coordinate; the ESP32 responds with the raw byte range. Per-page transfer at 1 MHz SPI is ≈2 ms; a single token's forward pass requires ≈64 pages of prototype data, so paged mode adds ≈130 ms per token, bringing total to ≈265 ms per token for a 131 KB student. Since Section 7 shows no such student outperforms the shipping model at BPC, we describe paged mode as an extension, not as a required part of the artifact.

9.4 What is not deployed

The shipping artifact is exclusively the pure-binary HDC-RWKV. The int8-output variant (F14, 148 KB) and continuous-decay variant (F15, 149 KB) are *research-only* artifacts: neither improves BPC over the shipping model, and both exceed the direct-fit budget. Deploying them would offer worse quality at worse cost. The architectural search of Section 8 therefore converges on a single deployable candidate; the choice of what to ship is empirical, not arbitrary.

10 Conclusion and future work

10.1 Summary of contributions

We introduced HDC-RWKV, a generative autoregressive language model whose every learnable parameter is bipolar at inference. The architecture combines HDC-style permutation for position encoding, an RWKV-style constant-cost-per-token recurrence, a continuous tanh-bounded state, and a prototype-similarity output, trained end-to-end with the clipped Straight-Through Estimator. The shipping artifact fits in 16,434 bytes and admits a pure bit-arithmetic inference kernel using the XOR/popcount identity (Equation 8), making it deployable on 1 MHz 8-bit silicon.

Beyond the artifact, we contributed a systematic empirical characterisation of the binary recurrent family's limits: a scale-independent BPC ceiling near 2.93 (F13); two independent component-relaxation ablations, on the output projection (F14) and on the decay gate (F15), neither of which breaks the ceiling; a controlled 4-point distillation α-sweep (F17) showing uniform regression under teacher signal; and, most informatively, the observation that the model self-organises to a bimodal $\{0, 1\}$ decay distribution when given continuous freedom (F16), evidence that the binary parameterisation is *preferred* at this scale, not imposed. Together these findings locate the residual bottleneck in the bipolar hidden state's information capacity, and rule out four natural alternatives.

10.2 Future work

State-side relaxations. The analysis in Section 8 names the specific class of experiments that could break the ceiling: modifications that change the state's effective bit budget rather than the components surrounding it. Multi-state variants, non-Markovian bipolar short-window memory, and higher-precision state activations (fp8, int4) each preserve most of the deployment-friendly properties and address the identified bottleneck.

Distillation follow-ups. The F17 sweep is single-seed and single-teacher. Two follow-ups would strengthen the result: (i) sweep distillation temperature T_s, which controls the sharpness of the target distribution, and (ii) test whether a bipolar-recurrent teacher (rather than a dense transformer) transfers better to a bipolar-recurrent student, matching the capacity-mismatch hypothesis of Stanton et al. [15].

On-silicon measurement. The 6502 firmware is planned but not measured; the $\approx$135 ms/token estimate of Section 9 is analytic. A measured firmware demonstration would upgrade Contribution 4 from "designed" to "demonstrated" and provide timing data for the paged-flash extension.

Broader impact. Language models at kilobyte scale enable always-on inference on hardware that pre-dates neural computing entirely. This has potential applications in embedded interfaces, historical computer emulation, and legibility of neural inference (a 16-KB artifact is small enough to inspect by hand). No new safety concerns arise beyond those of standard small-scale generative models trained on public domain corpora.

Reproducibility

All experiments in this paper use fixed random seed 1337. Source code, training scripts, and checkpoints are released alongside this paper [2]. Section 5 provides a hand-verifiable numerical trace on a toy instance.

References

1. Bengio, Y., Léonard, N., Courville, A.: Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint arXiv:1308.3432 (2013)
2. Bhattacharya, A., Gazi, N.: Wozformer: HDC-RWKV reference implementation and reproducibility artifacts. `https://github.com/elixpo/wozformer` (2026), source code, notebooks, checkpoints, and figures for this paper
3. Courbariaux, M., Bengio, Y., David, J.P.: BinaryConnect: Training deep neural networks with binary weights during propagations. In: Advances in Neural Information Processing Systems (NeurIPS) (2015)
4. Gu, A., Dao, T.: Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752 (2023)
5. Gu, A., Goel, K., Ré, C.: Efficiently modeling long sequences with structured state spaces. International Conference on Learning Representations (ICLR) (2022)
6. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531 (2015)
7. Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., Bengio, Y.: Quantized neural networks: Training neural networks with low precision weights and activations. In: Journal of Machine Learning Research (2018)
8. Kanerva, P.: Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. Cognitive Computation **1**(2), 139–159 (2009)
9. Karpathy, A.: The Unreasonable Effectiveness of Recurrent Neural Networks (2015), blog post; introduces the Tiny Shakespeare corpus
10. Peng, B., Alcaide, E., Anthony, Q., et al.: RWKV: Reinventing RNNs for the transformer era. arXiv preprint arXiv:2305.13048 (2023)
11. Plate, T.A.: Holographic reduced representations. IEEE Transactions on Neural Networks **6**(3), 623–641 (1995)
12. Qin, H., Gong, R., Liu, X., Bai, X., Song, J., Sebe, N.: Binary neural networks: A survey. Pattern Recognition **105**, 107281 (2020)
13. Rachkovskij, D.A., Kussul, E.M.: Binding and normalization of binary sparse distributed representations by context-dependent thinning. Neural Computation **13**(2), 411–452 (2001)
14. Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL) (2016)
15. Stanton, S., Izmailov, P., Kirichenko, P., Alemi, A.A., Wilson, A.G.: Does knowledge distillation really work? In: Advances in Neural Information Processing Systems (NeurIPS) (2021)